

CONTENTS

INTRODUCTION	1	CONTROL COMMUNICATION: REGISTER COMMUNICATION (WRITE)	4
Single-Cycle Data Communication	1		
Register Communication	1		
BLOCK DIAGRAM OF SYSTEM SETUP	2	CONTROL COMMUNICATION: REGISTER COMMUNICATION (READ)	5
SINGLE-CYCLE DATA COMMUNICATION (INCL. DATA VALIDATION)	3	REVISION HISTORY	6

INTRODUCTION

This application note provides example code in order to control iC-MB4 via SPI and perform basic *BiSS* communications as defined in the [BiSS C Protocol Description](#):

- Single-cycle data communication
- Register Communication

Single-Cycle Data Communication

Single-cycle data contains process data which is either transmitted from *BiSS* slave to master (sensor data) oder from *BiSS* master to slave (actuator data). In this application note an example code for an isochronous sensor data transmission is provided. Despite the process data itself, single-cycle data may contain additional data, e.g. **an error bit, a warning bit** or a sign-of-life counter (used in safety applications). Typically, the whole data content **except of the CRC bits** is considered by parameter **SCDLEN1** and the CRC polynomial (**SELCRCS1**, **SCRLEN1**) and CRC start value (**SCRCSTART1**) are configured separately. This enables iC-MB4 to perform CRC verification and indicate the result with its **SVALID** flags.

Once iC-MB4 is configured in accordance with the slave's characteristics (interface, data channel, frequencies), automatic equidistant sensor data acquisition can be started by setting **AGS = 1** as shown in SINGLE-CYCLE DATA COMMUNICATION (INCL. DATA VALIDATION). Note, the example includes a basic error handling to ensure that the received single-cycle data is valid. Additional error handling might be required depending on the specific application.

Register Communication

In each single-cycle data frame one control data bit is transmitted from master to slave (**CDM**) and from slave to master (**CDS**). The control data bits of several sin-

gle-cycle data frames form a control data frame. This control communication is suitable for:

- Register Communication
- *BiSS* Commands

Register Communication enables reading and writing of the sensor's registers. This feature is typically used to perform sensor calibration, reading the sensor's Electronic Data Sheet (EDS) or to read an internal temperature sensor register of the slave. Compared to single-cycle data communication, register communication is slower by design. Since only one control data bit is transmitted per single-cycle data frame in each direction, reading of a single slave register requires 32 single-cycle data frames.

In CONTROL COMMUNICATION: REGISTER COMMUNICATION (WRITE) one register value is written to the *BiSS* slave's register address 0x40. The [BiSS C Protocol Description](#) specifies that this register selects the *BiSS* slave's memory bank. Within the selected memory bank 64 registers can be read or written through register addresses 0x00 ... 0x3F. This memory concept provides up to 256 banks with 64 registers each (addresses 0x00 ... 0x3F). Typically, the first memory banks are used for the device's configuration parameters. Further memory banks may provide an EDS or additional memory for user content. Note, registers 0x40 ... 0x7F are static and always accessible independent of the selected memory bank.

In CONTROL COMMUNICATION: REGISTER COMMUNICATION (READ) two consecutive registers 0x7E and 0x7F (contain the Manufacturer ID) are read in one control data frame. Note, this sequential register access is not supported by every *BiSS* slaves. If sequential register access is not supported, registers 0x7E and 0x7F can be read in two separate control data frames.

BLOCK DIAGRAM OF SYSTEM SETUP

The following system setup is considered for the code examples. A microcontroller unit (MCU) is connected to the *BiSS* master iC-MB4 via SPI. The MCU configures iC-MB4 on startup according to the connected *BiSS* slaves and initiates the *BiSS* communication by setting

frame control parameters such as **FREQS**, **FREQAGS** and **AGS**. In this setup the *BiSS* slave is a 5 V encoder with an RS422 interface. Its data length (multiturn + singleturn + error bit + warning bit) is protected by a 6 bit CRC (polynomial: 0x43, start value: 0x00).

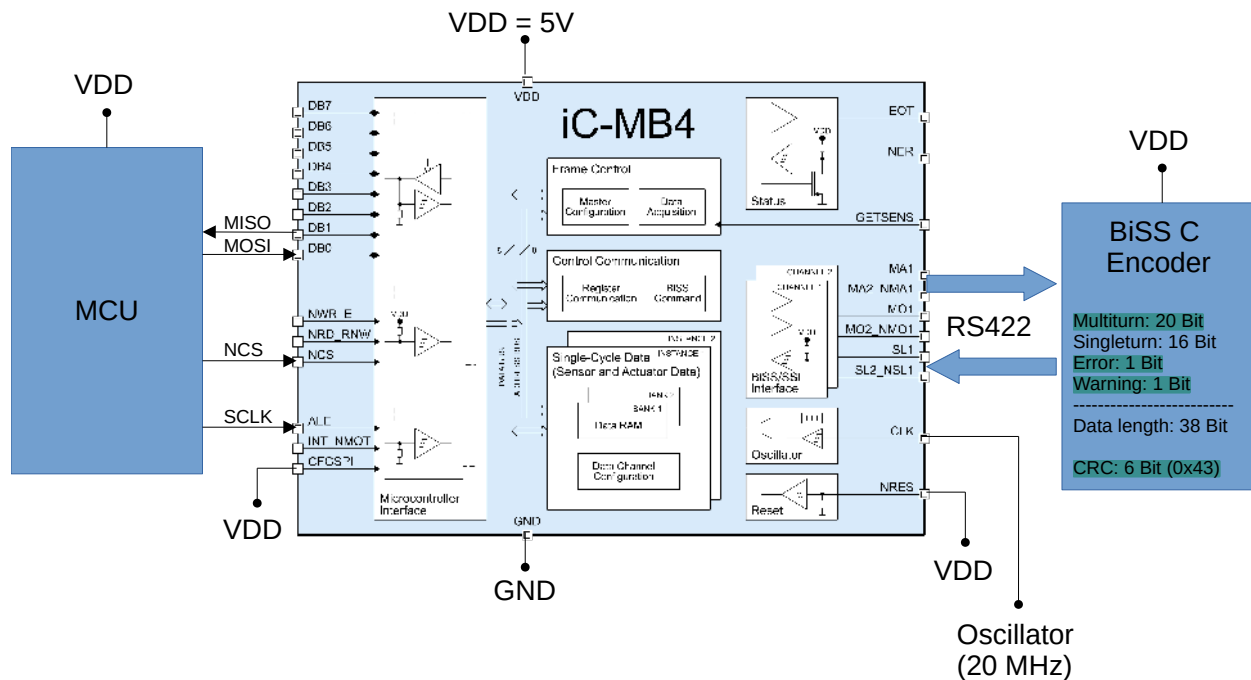


Figure 1: Block Diagram

SINGLE-CYCLE DATA COMMUNICATION (INCL. DATA VALIDATION)

```
1  //#####
2
3  //FUNCTIONS
4  //WriteMaster(address, value): Writes iC-MB4's register <address> with <value>
5  //ReadMaster(address): Reads iC-MB4's register <address>
6  //-----
7
8
9  //GENERAL SETUP
10 //-----
11
12 //BiSS/SSI Interface
13 WriteMaster(0xED, 0x01); //CFGCH1=0x01 (BiSS C)
14 WriteMaster(0xF5, 0x08); //CFGIF=0x02 (RS422)
15
16 //Single-Cycle Data: Data channel configuration
17 WriteMaster(0xC0, 0x65); //ENSCD1=1, SCDLEN1=0x25 (38 bit)
18 WriteMaster(0xC1, 0x06); //SELCRCS1=0x00, SCRCLEN1=0x06 (6-bit CRC polynomial 0x43)
19 WriteMaster(0xC2, 0x00); //SCRCSTART1(7:0)=0x00 (CRC start value)
20 WriteMaster(0xC3, 0x00); //SCRCSTART1(15:8)=0x00 (CRC start value)
21
22 //Frame Control: Master configuration
23 WriteMaster(0xE6, 0x04); //FREQS=0x04 (2 MHz)
24 WriteMaster(0xE8, 0x63); //FREQAGS=0x63 (10 kHz)
25
26
27 //VALIDATE AND READ SINGLE-CYCLE DATA
28 //-----
29
30 //Reset SVALID flags
31 WriteMaster(0xF1, 0x00);
32
33 //Start AGS
34 WriteMaster(0xF4, 0x01);
35 do{
36     //Read Status Information register 0xF0, wait for end of transmission EOT=1
37     do{
38         StatusInformationF0 = ReadMaster(0xF0);
39     }while ((StatusInformationF0 & 0x01) == 0);
40
41     //Read and reset SVALID flags in Status Information register 0xF1
42     StatusInformationF1 = ReadMaster(0xF1);
43     WriteMaster(0xF1, 0x00);
44
45     if ((StatusInformationF0 & 0x70) == 0x70)
46     {
47         //If Status ok but SVALID flag is not set, restart AGS
48         if ((StatusInformationF1 & 0x02) == 0)
49         {
50             WriteMaster(0xF4, 0x80); //BREAK
51             WriteMaster(0xF4, 0x01); //Restart AGS
52         }
53
54         //If Status ok and SVALID flag is set, read single-cycle data
55         else
56         {
57             SCDATA1_1=ReadMaster(0x00); //Least Significant Byte of SCDATA1
58             //...
59             SCDATA1_5=ReadMaster(0x04); //Most Significant Byte of SCDATA1
60         }
61     }
62     else
63         //If Status not ok, check data channel configuration
64 }while (1);
65
66 //#####
```

CONTROL COMMUNICATION: REGISTER COMMUNICATION (WRITE)

```
1  //#####
2
3  //FUNCTIONS
4  //WriteMaster(address, value): Writes iC-MB4's register <address> with <value>
5  //ReadMaster(address): Reads iC-MB4's register <address>
6  //-----
7
8
9  //GENERAL SETUP
10 //-----
11
12 //BiSS/SSI Interface
13 WriteMaster(0xED, 0x01); //CFGCH1=0x01 (BiSS C)
14 WriteMaster(0xF5, 0x08); //CFGIF=0x02 (RS422)
15
16 //Single-Cycle Data: Data channel configuration
17 WriteMaster(0xC0, 0x65); //ENSCD1=1, SCDLEN1=0x25 (38 bit)
18 WriteMaster(0xC1, 0x06); //SELCRC1=0x00, SCRCLEN1=0x06 (6-bit CRC polynomial 0x43)
19 WriteMaster(0xC2, 0x00); //SCRCSTART1(7:0)=0x00 (CRC start value)
20 WriteMaster(0xC3, 0x00); //SCRCSTART1(15:8)=0x00 (CRC start value)
21
22 //Frame Control: Master configuration
23 WriteMaster(0xE6, 0x04); //FREQS=0x04 (2 MHz)
24 WriteMaster(0xE8, 0x63); //FREQAGS=0x63 (10 kHz)
25
26
27 //CONTROL COMMUNICATION: WRITE SLAVE REGISTER
28 //Note: Register 0x40 of a BiSS slave contains the BankAddress (= selected memory bank)
29 //-----
30
31 //Configuration for writing slave register 0x40
32 WriteMaster(0xE2, 0xC0); //WNR=1 / REGADR=0x40
33 WriteMaster(0xE3, 0x00); //REGNUM=0x00 (only 1 slave registers)
34 WriteMaster(0xE4, 0x01); //CHSEL(1)=1 (selects channel 1 for control communication)
35 WriteMaster(0xE5, 0xC0); //CTS=1 (register access), REGVERS=1 (BiSS C), SLAVEID=0
36 WriteMaster(0x80, BankAddress); //RDATA=BankAddress
37
38 //Start register communication with slave
39 WriteMaster(0xF4, 0x09); //AGS=1, INSTR=4 (start control communication)
40 do
41 {
42     //value = ReadMaster(address)
43     MasterRegisterValue = ReadMaster(0xF4);
44 }
45 while (MasterRegisterValue > 1); //wait until INSTR (address 0xF4) is reset (AGS remains 1)
46
47 //Read status register
48 //Register communication successful if nREGERR=1 (bit3) and REGEND=1 (bit2)
49 MasterRegisterValue = ReadMaster(0xF0);
50
51 //#####
```

CONTROL COMMUNICATION: REGISTER COMMUNICATION (READ)

```
1  //#####
2
3  //FUNCTIONS
4  //WriteMaster(address, value): Writes iC-MB4's register <address> with <value>
5  //ReadMaster(address): Reads iC-MB4's register <address>
6  //-----
7
8
9  //GENERAL SETUP
10 //-----
11
12 //BiSS/SSI Interface
13 WriteMaster(0xED, 0x01); //CFGCH1=0x01 (BiSS C)
14 WriteMaster(0xF5, 0x08); //CFGIF=0x02 (RS422)
15
16 //Single-Cycle Data: Data channel configuration
17 WriteMaster(0xC0, 0x65); //ENSCD1=1, SCDLEN1=0x25 (38 bit)
18 WriteMaster(0xC1, 0x06); //SELCRC1=0x00, SCRCLEN1=0x06 (6-bit CRC polynomial 0x43)
19 WriteMaster(0xC2, 0x00); //SCRCSTART1(7:0)=0x00 (CRC start value)
20 WriteMaster(0xC3, 0x00); //SCRCSTART1(15:8)=0x00 (CRC start value)
21
22 //Frame Control: Master configuration
23 WriteMaster(0xE6, 0x04); //FREQS=0x04 (2 MHz)
24 WriteMaster(0xE8, 0x63); //FREQAGS=0x63 (10 kHz)
25
26
27 //CONTROL COMMUNICATION: READ SLAVE REGISTER
28 //Note: Registers 0x7E and 0x7F of a BiSS slave contain the Manufacturer ID
29 //Note: In this example two consecutive registers are read in one control communication.
30 //If the BiSS slave does not support sequential register accesses, the registers have to
31 //be read in two control communications by setting REGNUM=0 and REGADR accordingly.
32 //-----
33
34 //Configuration for reading slave registers 0x7E and 0x7F
35 WriteMaster(0xE2, 0x7E); //WNR=0 / REGADR=0x7E
36 WriteMaster(0xE3, 0x01); //REGNUM=0x01 (2 consecutive slave registers)
37 WriteMaster(0xE4, 0x01); //CHSEL(1)=1 (selects channel 1 for control communication)
38 WriteMaster(0xE5, 0xC0); //CTS=1 (register access), REGVERS=1 (BiSS C), SLAVEID=0
39
40 //Start slave register communication
41 WriteMaster(0xF4, 0x09); //AGS=1 / INSTR=4 (start control communication)
42 do
43 {
44     MasterRegisterValue = ReadMaster(0xF4);
45 }
46 while (MasterRegisterValue > 1); //wait until INSTR (address 0xF4) is reset (AGS remains 1)
47
48 //Read status register
49 //Register communication successful if nREGERR=1 (bit3) and REGEND=1 (bit2)
50 MasterRegisterValue = ReadMaster(0xF0);
51
52 //Read slave register values
53 SlaveRegisterValue0x7E = ReadMaster(0x80);
54 SlaveRegisterValue0x7F = ReadMaster(0x81);
55
56 //#####
```

iC-MB4 AN4

APPLICATION PROGRAMMING



Rev A1, Page 6/6

REVISION HISTORY

Rel.	Rel. Date*	Chapter	Modification	Page
A1	2022-03-18		Initial release	All

iC-Haus expressly reserves the right to change its products, specifications and related supplements (together the Documents). A Datasheet Update Notification (DUN) gives details as to any amendments and additions made to the relevant Documents on our internet website www.ichaus.com/DUN and is automatically generated and shall be sent to registered users by email.
Copying – even as an excerpt – is only permitted with iC-Haus' approval in writing and precise reference to source.

The data and predicted functionality is intended solely for the purpose of product description and shall represent the usual quality and behaviour of the product. In case the Documents contain obvious mistakes e.g. in writing or calculation, iC-Haus reserves the right to correct the Documents and no liability arises insofar that the Documents were from a third party view obviously not reliable. There shall be no claims based on defects as to quality and behaviour in cases of insignificant deviations from the Documents or in case of only minor impairment of usability.
No representations or warranties, either expressed or implied, of merchantability, fitness for a particular purpose or of any other nature are made hereunder with respect to information/specification resp. Documents or the products to which information refers and no guarantee with respect to compliance to the intended use is given. In particular, this also applies to the stated possible applications or areas of applications of the product.

iC-Haus products are not designed for and must not be used in connection with any applications where the failure of such products would reasonably be expected to result in significant personal injury or death (*Safety-Critical Applications*) without iC-Haus' specific written consent. Safety-Critical Applications include, without limitation, life support devices and systems. iC-Haus products are not designed nor intended for use in military or aerospace applications or environments or in automotive applications unless specifically designated for such use by iC-Haus.
iC-Haus conveys no patent, copyright, mask work right or other trade mark right to this product. iC-Haus assumes no liability for any patent and/or other trade mark rights of a third party resulting from processing or handling of the product and/or any other use of the product.

Software and its documentation is provided by iC-Haus GmbH or contributors "AS IS" and is subject to the ZVEI General Conditions for the Supply of Products and Services with iC-Haus amendments and the ZVEI Software clause with iC-Haus amendments (www.ichaus.com/EULA).

* Release Date format: YYYY-MM-DD